

Much Ado about Null

Keith Stockdale,
Senior Software Engineer



Asserting on nullptr

```
template<typename PointerType>
void TestNotNullOrAbort(
    const std::string& Message,
    const PointerType* Pointer)
{
    if (!Pointer)
    {
        throw AssertionException(Message);
    }
}
```

Asserting on nullptr

```
template<typename PointerType>
void TestNotNullOrAbort(
    const std::string& Message,
    const PointerType* Pointer)
{
    if (!Pointer)
    {
        throw AssertionException(Message);
    }
}

template<typename PointerType>
void TestNotNullOrAbort(const PointerType* Pointer)
{
    TestNotNullOrAbort(std::string{}, Pointer);
}
```

This overload is a problem...

```
template<typename PointerType>
void TestNotNullOrAbort(const PointerType* Pointer)
{
    TestNotNullOrAbort(std::string{}, Pointer);
}
```

... because this is valid code.

```
void SomeTestFunction()
{
    auto* Ptr = MakeAThing();
    TestNotNullOrAbort("Expected Ptr to be valid");
}
```

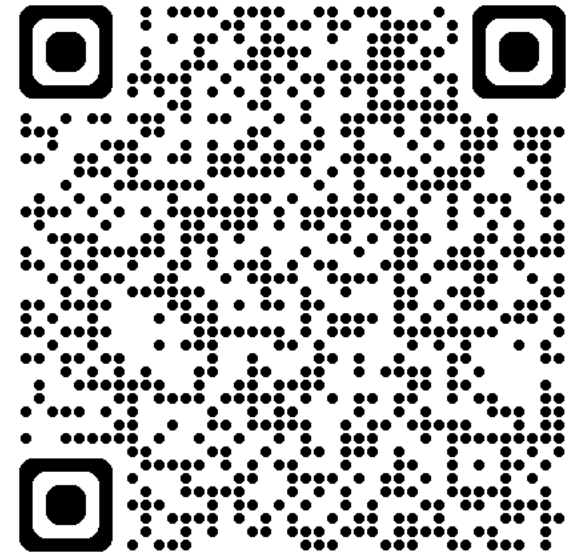
Thing 55

Good interfaces are:

- **Easy to use correctly.** People using a well-designed interface almost always use the interface correctly, because that's the path of least resistance. In a GUI, they almost always click on the right icon, button, or menu entry, because it's the obvious and easy thing to do. In an API, they almost always pass the correct parameters with the correct values, because that's what's most natural. With interfaces that are easy to use correctly, *things just work*.
- **Hard to use incorrectly.** Good interfaces anticipate mistakes people might make and make them difficult — ideally impossible — to commit. A GUI might disable or remove commands that make no sense in the current context, for example, or an API might eliminate argument-ordering problems by allowing parameters to be passed in any order.

Above all, remember that interfaces exist for the convenience of their users, not their implementers.

By [Scott Meyers](#) ↗



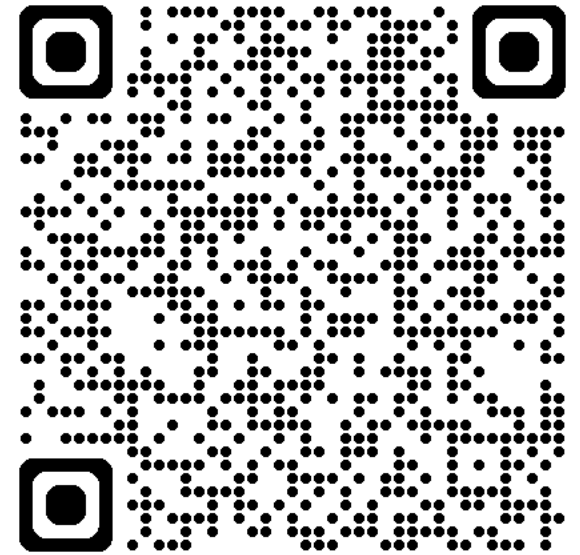
Thing 55

Good interfaces are:

- **Easy to use correctly.** People using a well-designed interface almost always use the interface correctly, because that's the path of least resistance. In a GUI, they almost always click on the right icon, button, or menu entry, because it's the obvious and easy thing to do. In an API, they almost always pass the correct parameters with the correct values, because that's what's most natural. With interfaces that are easy to use correctly, *things just work*.
- **Hard to use incorrectly.** Good interfaces anticipate mistakes people might make and make them difficult — ideally impossible — to commit. A GUI might disable or remove commands that make no sense in the current context, for example, or an API might eliminate argument-ordering problems by allowing parameters to be passed in any order.

Above all, remember that interfaces exist for the convenience of their users, not their implementers.

By [Scott Meyers](#) ↗



Concepts to the rescue!

```
template<typename PointerType>
void TestNotNullOrAbort(const PointerType* Pointer)
{
    TestNotNullOrAbort(std::string{}, Pointer);
}
```

Concepts to the rescue!

```
template<typename PointerType>
    requires (!TIsCharTypeV<PointerType>)
void TestNotNullOrAbort(const PointerType* Pointer)
{
    TestNotNullOrAbort(std::string{}, Pointer);
}
```

This code no longer compiles 😊

```
void SomeTestFunction()
{
    auto* Ptr = MakeAThing();
    TestNotNullOrAbort("Expected Ptr to be valid");
}
```

We must pass the actual pointer now

```
void SomeTestFunction()
{
    auto* Ptr = MakeAThing();

    TestNotNullOrAbort("Expected Ptr to be valid", Ptr);
}
```

Back to our test

```
void SomeTestFunction()
{
    auto* Ptr = MakeAThing();

    // Ptr could be null
    TestNotNullOrAbort("Expected Ptr to be valid", Ptr);
    // Ptr is definitely not null :)
}
```

Back to our test

```
void SomeTestFunction()
{
    auto* Ptr = MakeAThing();

    // Ptr could be null
    TestNotNullOrAbort("Expected Ptr to be valid", Ptr);
    // Ptr is definitely not null :)

    // Some more complicated test code...
}
```

Back to our test

```
void SomeTestFunction()
{
    auto* Ptr = MakeAThing();

    // Ptr could be null
    TestNotNullOrAbort("Expected Ptr to be valid", Ptr);
    // Ptr is definitely not null :)

    // Some more complicated test code...

    Ptr->DoTheThing();
}
```

Back to our test

```
void SomeTestFunction()
{
    auto* Ptr = MakeAThing();

    // Ptr could be null
    TestNotNullOrAbort("Expected Ptr to be valid", Ptr);
    // Ptr is definitely not null :)

    // Some more complicated test code...

    // Are we sure that Ptr still can't be null?
    Ptr->DoTheThing();
}
```

Can we do better?

```
void SomeTestFunction()
{
    auto* Ptr = MakeAThing();

    // Ptr could be null
    TestNotNullOrAbort("Expected Ptr to be valid", Ptr);
    // Ptr is definitely not null :)

    // Some more complicated test code...

    // Are we sure that Ptr still can't be null?
    Ptr->DoTheThing();
}
```

We could use const...

```
void SomeTestFunction()
{
    // Stop any further assignment
    auto* const Ptr = MakeAThing();

    // Ptr could be null
    TestNotNullOrAbort("Expected Ptr to be valid", Ptr);
    // Ptr is definitely not null :)

    // Some more complicated test code...

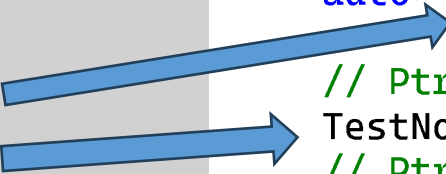
    // Yes, we are sure that Ptr can't be null
    Ptr->DoTheThing();
}
```

This introduces complexity

```
void SomeTestFunction()
{
    // Stop any further assignment
    auto* const Ptr = MakeAThing();
    // Ptr could be null
    TestNotNullOrAbort("Expected Ptr to be valid", Ptr);
    // Ptr is definitely not null :)

    // Some more complicated test code...

    // Yes, we are sure that Ptr can't be null
    Ptr->DoTheThing();
}
```



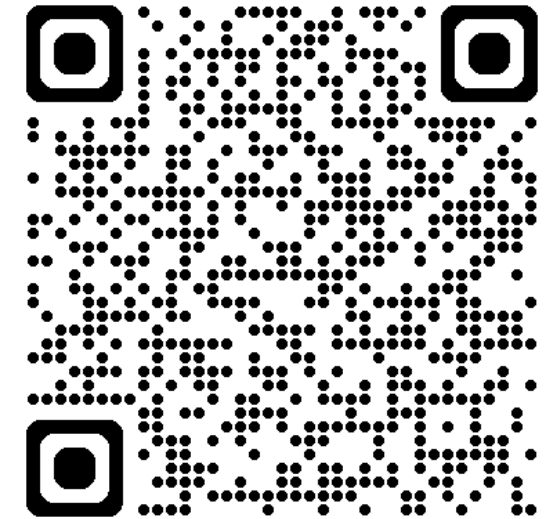
Simple Made Easy



Strange Loop
Sept 19-20, 2011
<https://thestrangeloop.com>

Simple

- One fold/braid
 - One role
 - One task
 - One concept
 - One dimension
- But not
 - One instance
 - One operation
 - About lack of interleaving, not cardinality
 - *Objective*



But there is a better way...

```
void SomeTestFunction()
{
    auto* Ptr = MakeAThing();

    // Ptr could be null
    auto& Ref = TestNotNullOrAbort("Expected Ptr to be valid", Ptr);
    // Use Ref from here :)

    // Some more complicated test code...

    // Definitely not null :D
    Ref.DoTheThing();
}
```

The implementation is a little harder...

```
template<typename PointerType>
    requires std::is_pointer_v<std::remove_reference_t<PointerType>>
decltype(auto) TestNotNullOrAbort(const std::string& Message, PointerType&& Pointer)
{
    if (!Pointer)
    {
        throw AssertionException(Message);
    }
}
```

The implementation is a little harder...

```
template<typename PointerType>
    requires std::is_pointer_v<std::remove_reference_t<PointerType>>
decltype(auto) TestNotNullOrAbort(const std::string& Message, PointerType&& Pointer)
{
    if (!Pointer)
    {
        throw AssertionException(Message);
    }
    else
    {
        if constexpr (TIsVoidPointer<PointerType>)
        {
            return;
        }
        else
        {
            return *Pointer;
        }
    }
}
```

Luckily, we can sanity check!

```
template<typename T, typename Expected>
concept CValidateNotNullOrAbortReturnType = requires(FFakeAutomationTest& Test, T&& Ptr)
{
    { Test.TestNotNullOrAbort(Forward<T>(Ptr)) } ->
        RareConcepts::TSameAs<Expected>;
};
```

Luckily, we can sanity check!

```
template<typename T, typename Expected>
concept CValidateNotNullOrAbortReturnType = requires(FFakeAutomationTest& Test, T&& Ptr)
{
    { Test.TestNotNullOrAbort(Forward<T>(Ptr)) } ->
        RareConcepts::TSameAs<Expected>;
};
```

```
static_assert(CValidateNotNullOrAbortReturnType<const int32*, const int32*>);
static_assert(CValidateNotNullOrAbortReturnType<int32*, int32*>);
static_assert(CValidateNotNullOrAbortReturnType<const void*, void>);
static_assert(CValidateNotNullOrAbortReturnType<void*, void>);
```

```
static_assert(CValidateNotNullOrAbortReturnType<const int32*&, const int32*>);
static_assert(CValidateNotNullOrAbortReturnType<int32*&, int32*>);
static_assert(CValidateNotNullOrAbortReturnType<const void*&, void>);
static_assert(CValidateNotNullOrAbortReturnType<void*&, void>);
```

A nice consequence...

```
template<typename T, typename Expected>
concept CValidateNotNullOrAbortReturnType = requires(FFakeAutomationTest& Test, T&& Ptr)
{
    { Test.TestNotNullOrAbort(Forward<T>(Ptr)) } ->
        RareConcepts::TSameAs<Expected>;
};
```

```
static_assert(CValidateNotNullOrAbortReturnType<const int32*, const int32*>);
static_assert(CValidateNotNullOrAbortReturnType<int32*, int32*>);
static_assert(CValidateNotNullOrAbortReturnType<const void*, void>);
static_assert(CValidateNotNullOrAbortReturnType<void*, void>);
```

```
static_assert(CValidateNotNullOrAbortReturnType<const int32*&, const int32*>);
static_assert(CValidateNotNullOrAbortReturnType<int32*&, int32*>);
static_assert(CValidateNotNullOrAbortReturnType<const void*&, void>);
static_assert(CValidateNotNullOrAbortReturnType<void*&, void>);
```

```
static_assert(!CValidateNotNullOrAbortReturnType<decltype(nullptr), void>);
```

Thanks!

